Pre-Phase 4 Remediation Plan: Migration of Factory Ledgers to the Distributed Index-Leaf Paradigm

1. Architectural Preamble: The Imperative of the Distributed Graph

The evolution of the Stela repository, specifically within the php-nuke-ce context, represents a fundamental shift in systems architecture from static, monolithic file retention to a dynamic, cryptographically verifiable graph topology. This transition, codified as the **Distributed Index-Leaf Filing Paradigm**, is not merely an operational convenience but a survival mechanism for the repository's long-term integrity. As the system prepares for **Phase 4: Graph Walk Validation**, the structural anomalies present within the "Factory" subsystems—specifically the agent, task, and skill ledgers—constitute a critical blocking factor. These surfaces (opt/_factory/AGENTS.md, TASKS.md, SKILLS.md) currently operate under an obsolete "append-only monolith" model that is incompatible with the Quartet Filing Doctrine and the automated verification logic of the incoming graph walker.

This report formulates a rigorous, pre-Phase 4 remediation plan. It synthesizes the forensic results of **DP-OPS-0073** [1], which successfully extended the leafing paradigm to the compilation tier, and applies those lessons to the factory layer. The objective is to deconstruct the "hybrid monsters" residing in opt/_factory/—files that conflate static governance (descriptive text) with dynamic history (logs)—and reorganize them into a distributed architecture where roots serve as pointers, history resides in immutable leaves, and continuity is guaranteed by cryptographic "Test Candidate Sentinels."

The urgency of this remediation cannot be overstated. Phase 4 assumes a repository state where every significant artifact is a node in a traversable graph, linked by explicit previous pointers back to a genesis state.[1] The current factory files, being unstructured text blobs, are opaque to this logic. Without the intervention detailed herein, the deployment of the Graph Walk Validator will result in immediate, systemic failure notifications, paralyzing the CI/CD pipeline and forcing a manual rollback. This plan provides the blueprint for a "Split-State" migration that preserves historical fidelity while enforcing the new unified schema.

---

2. Forensic Analysis of DP-OPS-0073 Findings

To engineer a successful migration for the factory ledgers, one must first dissect the operational precedents established by **DP-OPS-0073**, certified on February 18, 2026.[1] This

Data Packet (DP) was not simply a maintenance cycle; it was a constitutional amendment that validated the feasibility of the Distributed Index-Leaf Filing Paradigm in a high-churn environment.

2.1 The Certification of Atomic Snapshotting

The primary achievement of DP-OPS-0073 was the successful upgrade of the compile binary to emit immutable manifest snapshot leaves.[1] Prior to this, manifest generation was ephemeral; the system overwrote ops/lib/manifests/*.md in place, destroying the history of the context. DP-OPS-0073 introduced a mechanism where every compilation event generates a discrete, schema-conformant leaf in archives/manifests/ (e.g., compile-2026-02-18T203136-b9d161f7.md) before updating the runtime surface.[1]

This result is pivotal for the factory migration. It proves that the system can sustain a "dual-tier" architecture:

1. **The Runtime Tier:** Mutable files used by the system (e.g., the current CONTEXT.md or the future factory HEAD pointer).
2. **The Archive Tier:** Immutable leaves that preserve the exact state at the moment of creation.

The success of the compile upgrade confirms that high-frequency operations—such as compiling the context or harvesting an agent—can generate persistent artifacts without bloating the root directory structure. The integration of "atomic temp output" ensures that partial writes do not corrupt the chain [1], a logic pattern that must be replicated in the factory harvester scripts.

2.2 The Retirement of Transitional Logic

A significant finding from the DP-OPS-0073 RESULTS is the retirement and deletion of ops/lib/scripts/retroleaf.sh, removing 685 lines of technical debt.[1] This script was a "bridge" tool used in Phase 2 to parse monolithic history into leaves retroactively. Its retirement signals that the repository has moved into a **"Forward-Only" Mode**.

Implication for Factory Migration: We cannot rely on the now-deleted retroleaf.sh to migrate the factory ledgers. A new, specialized migration utility (or a "Factory Retroleaf" variant) must be developed specifically for AGENTS.md, TASKS.md, and SKILLS.md. This utility must handle the unique internal structure of these files, which differ significantly from the SoP.md and PoW.md structures processed in Phase 2. The factory files contain dual log sections ("Candidate Log" and "Promotion Log") [1] interwoven with narrative instructions, requiring a more sophisticated parsing logic than the linear ledgers of Phase 2.

2.3 Schema Integrity Verification

The introduction of tools/lint/schema.sh (LINT-12) in DP-OPS-0073 successfully validated 51

files against the Unified Schema.[1] This schema requires four specific front-matter keys: trace_id, packet_id, created_at, and previous.[1]

The successful validation of these files confirms that the schema is robust enough to handle diverse artifact types. However, it also imposes a hard constraint on the factory migration: **Every new factory leaf must strictly adhere to this schema.** The factory scripts (ops/lib/scripts/agent.sh, etc.) must be refactored to inject these specific keys during the harvest and promotion cycles. Failure to do so will cause the new LINT-12 gate to block the remediation PR.

2.4 The Risk of Schema Drift

The DP-OPS-0073 analysis highlighted a "Schema Drift Risk" where manifest archive leaves might diverge from the active runtime manifests.[1] This was mitigated by extending validation logic. In the context of the factory, this risk is amplified. If the format of the "Candidate Log" inside the leaf differs from the format expected by the promote command, the toolchain will break. The remediation plan must ensure strict parity between the *output* of the harvest command and the *input* of the promote command within the new leaf structure.

---

3. Structural Pathology of the Factory Scope

The opt/_factory/ directory represents the final bastion of the "Old Regime"—the monolithic, file-centric architecture that predates the Distributed Index-Leaf Filing Paradigm. An analysis of the repository dump [1] reveals that this directory operates in direct violation of the Quartet Filing Doctrine and the Separation of Concerns principle.

3.1 The "Hybrid Monster" Anti-Pattern

The three core files—AGENTS.md, TASKS.md, and SKILLS.md—are currently "Hybrid Monsters." They attempt to serve three distinct, conflicting functions simultaneously, leading to inevitable contention and bloat.[1]

3.1.1 Function 1: The Manual (Static Governance)

Each file contains extensive descriptive text: "Operator Guidance," "Harvest Engine Workflow," and "Promotion Packet Templates".[1] This is static content. It defines *how* the system works. Under the Filing Doctrine, static documentation belongs in docs/, specifically docs/ops/specs/. Keeping it in opt/ violates the principle that opt/ is for "Isolated JIT-only content" or definition pointers, not operator manuals.

3.1.2 Function 2: The Registry (Inventory)

The files act as lists of available tools. However, the file index [1] reveals the existence of a parallel,

authoritative registry system in docs/:

- docs/ops/registry/AGENTS.md
- docs/ops/registry/SKILLS.md
- docs/ops/registry/TASKS.md

This duplication is a violation of the **Single Source of Truth (SSoT)** rule defined in PoT.md.[1] If opt/_factory/AGENTS.md lists an agent that is not in docs/ops/registry/AGENTS.md, which is true? The remediation plan must resolve this by stripping the registry function from the factory roots entirely, deferring to the docs/ registries as the SSoT.

3.1.3 Function 3: The Log (Dynamic History)

Finally, these files serve as mutable logs. They contain "Candidate Log" and "Promotion Log" sections where scripts append new entries.[1] This is the most dangerous function. As the system scales, these logs grow indefinitely.

- **The Git Conflict:** In a distributed team, two operators promoting different agents simultaneously will cause a merge conflict in AGENTS.md, as both attempt to append to the bottom of the file.
- **The Context Hazard:** When an LLM or human operator loads AGENTS.md to find the "Harvest Engine Workflow," they are forced to load the entire history of every agent ever created. This wastes token context window and obscures the relevant instructions.

3.2 The Traversal Blockage (Phase 4 Barrier)

The most critical pathology is the "Traversal Blockage." Phase 4 introduces a Graph Walk Validator.[1] This validator expects every file in the governance scope to be a node in a directed acyclic graph (DAG). It looks for a previous pointer to traverse backward.

Currently, opt/_factory/AGENTS.md is a flat text file. It has no previous pointer. It has no trace_id. It is a dead end. If the Graph Walk Validator were deployed today, it would flag opt/_factory as a "Systemic Continuity Failure," causing the repository integrity check to fail. The validator cannot parse a previous link from a file that is 90% human-readable documentation and 10% append-only log lines.

3.3 The Disconnected Lifecycle

The current lifecycle scripts (ops/lib/scripts/agent.sh, etc.) are hardcoded to interact with these monoliths via simple text appending (cat >>). They do not perform cryptographic linking. They do not check if the previous entry hash matches. This means the current history is "fragile"—a line can be deleted from the middle of AGENTS.md without invalidating the integrity of subsequent entries. The Distributed Index-Leaf Paradigm requires a "Tamper-Evident" chain, where deleting an entry breaks the hash link of the next leaf.[1] The current factory structure supports none of this.

4. The Remediation Strategy: Separation of Concerns

To resolve the structural pathologies identified above, the remediation plan employs a strategy of **Radical Separation**. We must explode the monolithic files into their constituent functional parts and relocate them to their appropriate homes within the Quartet topology.

4.1 Dimension 1: Relocating Descriptive Text (The Constitution)

The static governance text found in the factory ledgers is effectively the "Constitution" of the factory. It describes the laws of Agent creation, Task definition, and Skill acquisition.

- **Destination:** docs/ops/specs/factory/.
- **Action:** We will extract the "Harvest Engine Workflow," "Doctrine," and "Template" sections from the root files.
- **New Artifacts:**
  - docs/ops/specs/factory/agents.md: Will contain the rules for R-AGENT harvesting.
  - docs/ops/specs/factory/tasks.md: Will contain the rules for B-TASK orchestration.
  - docs/ops/specs/factory/skills.md: Will contain the rules for S-LEARN usage.
- **Precedent:** This mirrors the remediation of PoW.md in DP-OPS-0072 [1], where operational guidance was moved to docs/ops/specs/surfaces/pow.md to allow the root file to become a pure pointer.

4.2 Dimension 2: Converting Roots to Pointers (The Place Marker)

Once the text is removed, the files opt/_factory/AGENTS.md, TASKS.md, and SKILLS.md must be rewritten. They will no longer hold content. Instead, they will serve as **HEAD Pointers** or **Place Markers**.[1]

- **New State:** A single-line file containing a relative path.
- **Target:** The most recent leaf file in the archives/ chain.
- **Example Content:** archives/surfaces/factory/agents/AGENTS-2026-02-18-b9d161f7.md
- **Behavior:** When the agent.sh promote script runs, it will read this line to find the *previous* state, generate a new leaf, and then overwrite this single line with the path to the new leaf. This creates a mutable pointer to an immutable history.

4.3 Dimension 3: Atomizing Dynamic Logs (The History)

The "Candidate Log" and "Promotion Log" sections are the actual history of the system. These must be converted into **Immutable Leaves**.

- **Destination:** archives/surfaces/factory/{agents,tasks,skills}/.
- **Unified Schema:** Each leaf must carry the unified front-matter (trace_id, packet_id, etc.).[1]
- **Granularity:** Unlike the monolithic append model, a leaf is generated for *every single transaction*. If an operator harvests a candidate, a leaf is born. If they promote it, another

leaf is born.
- **Chain Resolution:** The plan requires merging the separate "Candidate" and "Promotion" logs into a single chronological chain per factory type. Maintaining two separate chains for Agents (one for candidates, one for promotions) introduces unnecessary complexity in pointer management. A unified "Agent Ledger" chain, where the operation field distinguishes the event type, is the superior architectural choice.

---

5. The "Test Candidate" Sentinel Pattern (Pointer Continuity)

The user query explicitly mandates the inclusion of **"test candidates which will remain to maintain an link/opening"**. This requirement addresses a subtle but critical failure mode known as the **Genesis Problem** or the **Empty Set Paradox** in graph-based file systems.

5.1 The Genesis Problem

If we wipe opt/_factory/AGENTS.md and turn it into a pointer, what does it point to initially?

- If it points to nothing (empty file), the first run of the agent.sh script will fail because it cannot resolve PREVIOUS. The logic cat opt/_factory/AGENTS.md will return null.
- If we define PREVIOUS as (origin), we break the uniformity of the validation logic, which expects a file path.
- More importantly, if we migrate *historical* data, we need an anchor point for the oldest historical entry to link back to.

5.2 The Sentinel Solution

The solution is to mint a "Test Candidate" or "Sentinel Leaf" effectively the "Genesis Block" of the chain. This leaf is manually created during the remediation DP. It serves no operational purpose other than to exist, providing a valid file path for the HEAD pointer and a valid previous target for the first actual log entry.

5.3 Sentinel Specifications

We will create three Sentinel Leaves, one for each factory type.

File: archives/surfaces/factory/agents/AGENTS-GENESIS.md

trace_id: stela-GENESIS-0000 packet_id:

DP-OPS-0074 created_at: 2026-02-18T21:00:00Z previous: (origin)

Agent Factory Genesis Sentinel

Transaction Metadata

- Operation: SENTINEL
- Actor: Integrator

Log Payload

- Note: This sentinel maintains pointer continuity for the Distributed Index chain.
- Status: ACTIVE

Linkage: The root file opt/_factory/AGENTS.md will be initialized with the single line:

archives/surfaces/factory/agents/AGENTS-GENESIS.md

This ensures that:

1. **Continuity:** The link is "maintained/open" as requested.
2. **Resolution:** Scripts reading the root always get a valid path.
3. **Validation:** The Graph Walk Validator can successfully traverse from HEAD to this file, see previous: (origin), and terminate the walk successfully (Green State).

---

6. Pre-Phase 4 Remediation Plan (DP-OPS-0074)

This section details the step-by-step execution plan for **DP-OPS-0074: Factory Ledger Migration**. This plan is formulated to be atomic: the repository transitions from Monolith to Graph in a single commit sequence to avoid "Split-Brain" states.

6.1 Phase A: Infrastructure Preparation

Before touching the content, we must establish the physical structures required by the new paradigm.

1. **Directory Creation:**
    - mkdir -p docs/ops/specs/factory
    - mkdir -p archives/surfaces/factory/agents
    - mkdir -p archives/surfaces/factory/tasks
    - mkdir -p archives/surfaces/factory/skills
    - *Note:* Ensure .gitkeep files are placed in these new archive directories to ensure they

are tracked even if empty initially.
2. **Sentinel Minting:**
    - Create the three Genesis Sentinel files as defined in Section 5.3.
    - Validate they pass the tools/lint/schema.sh check immediately upon creation.

6.2 Phase B: Static Content Relocation

We now extract the "Constitution" from the "Hybrid Monsters."

1. **Extract AGENTS.md:**
    - Copy the "Harvest Engine Workflow" and "Promotion Packet Template" sections.
    - Create docs/ops/specs/factory/agents.md.
    - *Refinement:* Update the text in the new spec to reference the new tooling behavior (e.g., "The system generates a leaf" instead of "The system appends to the ledger").
2. **Extract TASKS.md & SKILLS.md:** Repeat the process for the Task and Skill ledgers.
3. **Update Manuals:** Update docs/MANUAL.md and opt/_factory/INDEX.md to point to these new locations as the authoritative source for factory usage instructions.

6.3 Phase C: Retroactive Log Migration (The Factory Retroleaf)

We must preserve the existing history. We cannot simply delete the "Candidate Log" sections.

1. **Development of ops/lib/scripts/factory-retroleaf.sh:**
    - This temporary script will parse the legacy monolithic files.
    - **Logic:** It scans for lines starting with - YYYY-MM-DD.
    - **Transformation:** For each entry, it generates a new leaf file in archives/surfaces/factory/....
    - **Chaining:** It links the first extracted entry to the Genesis Sentinel. It links the second entry to the first, and so on.
    - **Metadata:** It extracts the created_at timestamp from the log line itself to preserve historical accuracy. trace_id for these retro-entries will be generated as stela-MIGRATION-<HASH> to indicate they were post-processed.
2. **Execution:**
    - Run the script against the three factory files.
    - Verify that a chain of leaves now exists in archives/surfaces/factory/.
    - Capture the filename of the *last* (most recent) leaf generated for each type.

6.4 Phase D: Root Pointer Conversion

This is the destructive step. Once history is safe in the archives, we nuke the monoliths.

1. **Overwrite opt/_factory/AGENTS.md:** Replace the entire content with the path to the *last* leaf generated in Phase C.
2. **Overwrite TASKS.md & SKILLS.md:** Repeat for Tasks and Skills.
3. **Result:** The factory roots are now 1-line HEAD pointers.

6.5 Phase E: Tooling Alignment (The Split-Write Update)

The lifecycle scripts must be updated to function in this new world. We must refactor ops/lib/scripts/agent.sh, task.sh, and skill.sh.

The New "Promote" Logic:

Currently, promote does: cat entry >> opt/_factory/AGENTS.md.

The new logic must be:

Bash

```
# 1. Resolve Previous
PREVIOUS_LEAF="$(cat opt/_factory/AGENTS.md)"

# 2. Generate New Leaf Path
NEW_LEAF="archives/surfaces/factory/agents/AGENTS-$(date +%F)-$(short_hash).md"

# 3. Write Leaf (Unified Schema + Log Payload)
cat <<EOF > "$NEW_LEAF"
---
trace_id: ${STELA_TRACE_ID}
packet_id: ${DP_ID}
created_at: $(date -u +%FT%TZ)
previous: ${PREVIOUS_LEAF}
---
# Agent Promotion Entry
... (Log Payload)...
EOF

# 4. Update Pointer
echo "$NEW_LEAF" > opt/_factory/AGENTS.md
```

This logic ensures that every future operation extends the cryptographic chain initiated by the Sentinel and the Migration script.

---

7. Verification and Integrity: The Phase 4 Bridge

The ultimate success criterion for this remediation is the readiness for Phase 4. How do we

prove we are ready?

7.1 Manual Graph Walk

Before closing DP-OPS-0074, we will execute a manual graph walk.

1. Read opt/_factory/AGENTS.md.
2. Open the file it points to.
3. Read the previous line.
4. Open that file.
5. Repeat until previous: (origin) is reached (which should be the Genesis Sentinel).

If this process completes without a "File Not Found" error, the chain is valid.

7.2 LINT-12 Enforcement

We will run tools/lint/schema.sh against the new archives/surfaces/factory/ directories. This ensures that our factory-retroleaf.sh script and our updated lifecycle scripts are correctly applying the Unified Schema.

7.3 Registry Consistency Check

We will run tools/lint/factory.sh.[1] Since we stripped the registry function from the log files, we must ensure that the docs/ops/registry/ files are accurate. The linter might need adjustment if it previously cross-referenced the monoliths. It should now only cross-reference the *files* in opt/_factory/agents/ against the docs/ registry, completely ignoring the opt/_factory/AGENTS.md pointer file.

---

8. Data Tables and Schemas

Table 1: Factory Component Migration Map

Component	Current Location (Monolith)	New Location (Distributed)	Role
Operational Specs	opt/_factory/AGENTS.md (Body)	docs/ops/specs/factory/agents.md	Static Constitution
Historical Logs	opt/_factory/AGENTS.md (Append)	archives/surfaces/factory/agents/*.md	Immutable History
Current State	opt/_factory/AGENT	opt/_factory/AGENT	Navigation Head

	S.md (File)	S.md (Pointer)	
Continuity Anchor	N/A (Does not exist)	.../AGENTS-GENESIS.md	Cryptographic Root

Table 2: Unified Factory Leaf Schema (Target State)

Field	Value Source	Purpose
trace_id	STELA_TRACE_ID (Env) or stela-MIGRATION	Session correlation and debugging.
packet_id	DP-OPS-XXXX	Provenance tracking.
created_at	System Clock (UTC)	Chronological ordering.
previous	cat opt/_factory/TYPE.md	Cryptographic verification (Graph Walk).
operation	harvest, promote, sentinel	Distinguishes transaction type in the unified chain.

---

9. Conclusion

The "Hybrid Monster" state of the factory ledgers is an untenable technical debt that endangers the transition to Phase 4. By executing this remediation plan, we align the final holdouts of the opt/ directory with the Quartet Filing Doctrine.

The transformation is three-fold:

1. **Sanitization:** The opt/_factory roots become clean, single-line pointers, adhering to the "Place Marker" philosophy.
2. **Preservation:** Historical data is rescued from the monoliths and crystallized into immutable, verifiable leaves.
3. **Continuity:** The introduction of "Test Candidate Sentinels" solves the Genesis Problem, ensuring that the pointer chain is unbroken from the moment of inception.

This plan moves the repository from a state of "fragile accumulation" to "cryptographic certainty." Upon the completion of DP-OPS-0074, the Stela repository will present a unified,

traversable graph surface to the Phase 4 validator, securing the integrity of the system for all future operations.

Immediate Recommendations

1. **Approve DP-OPS-0074** with the scope defined in Section 6.
2. **Authorize the deletion** of legacy log content from opt/_factory once the retroleaf migration is verified.
3. **Enforce the new schema** via tools/lint/schema.sh immediately upon transition.

This concludes the remediation formulation. The path to Phase 4 is clear.

Works cited

1. DP-OPS-0073-RESULTS.md